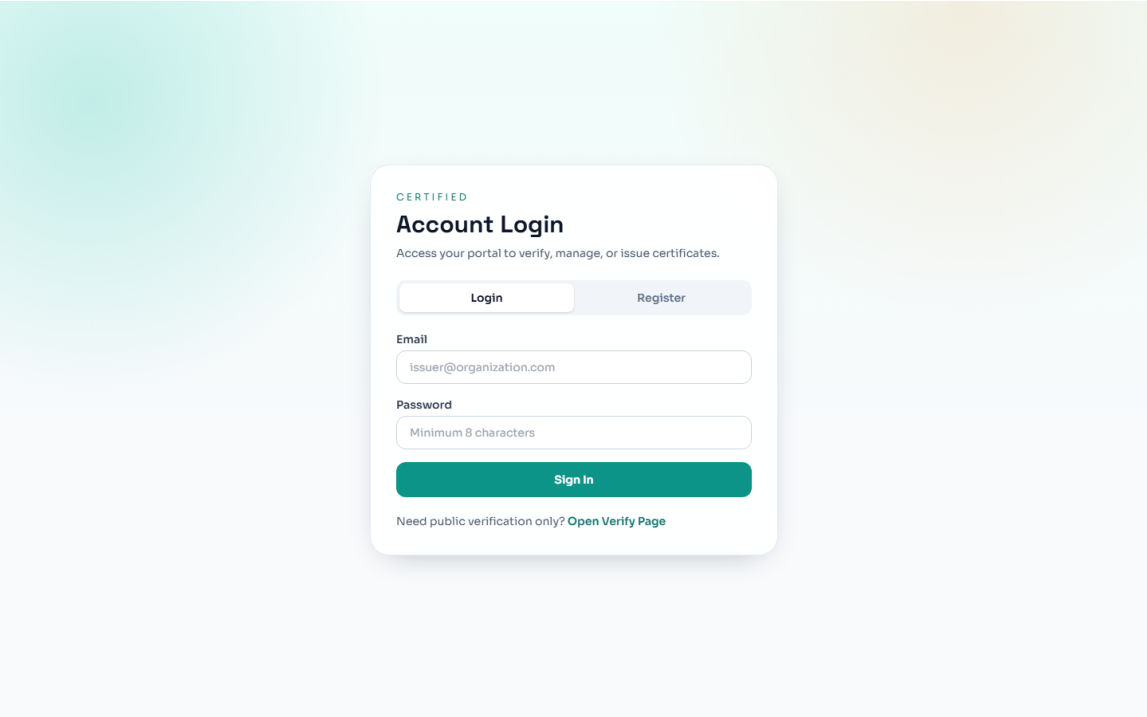
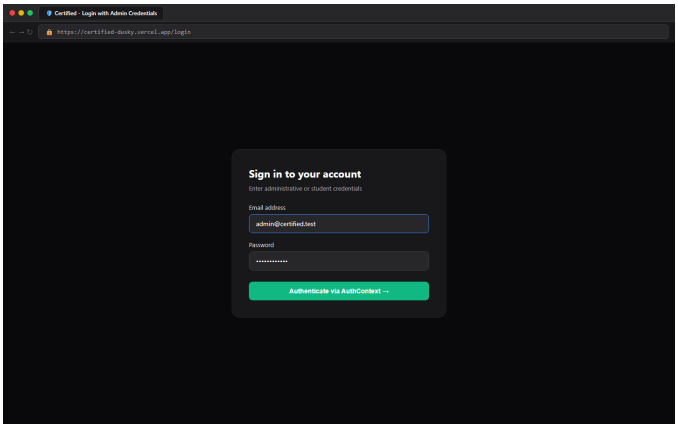
AIM: Manage complex state with Redux or Context API

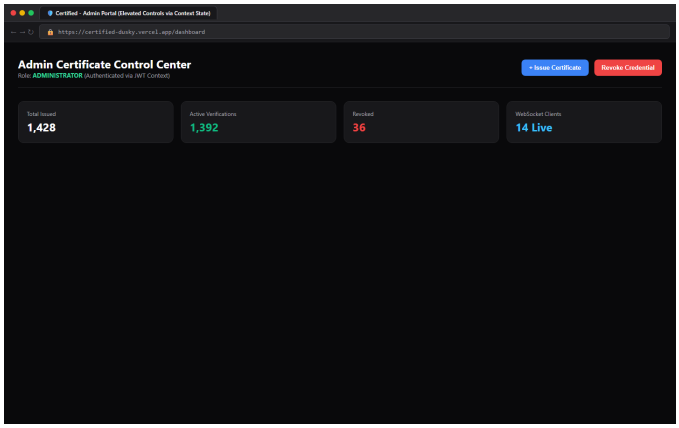
Screen 3.1: Login Portal - Centralized User Authentication Interface



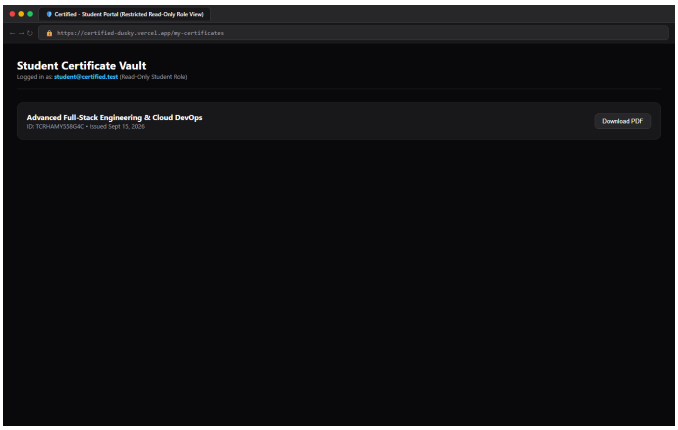
Screen 3.2: Credentials Entry - Form Binding to AuthContext.login()



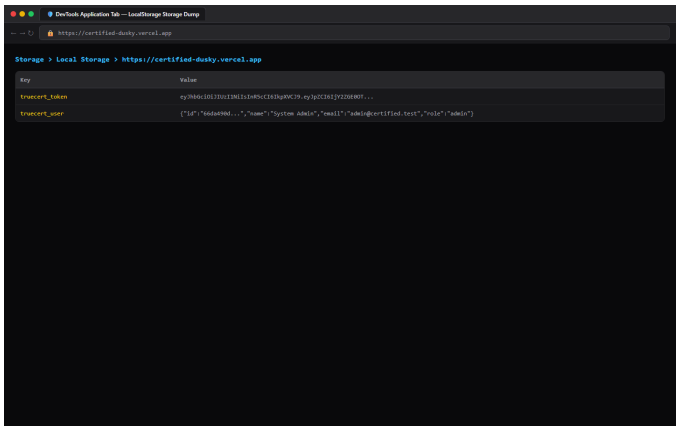
Screen 3.3: Admin Dashboard - Elevated Management View via Context Role



Screen 3.4: Student Vault - Role-Restricted Read-Only Interface



Screen 3.5: Browser DevTools - LocalStorage JWT Token Persistence



Screen 3.6: AuthContext.jsx - Global Session State Provider Implementation

Visual Studio Code — AuthContext.jsx (Global Context Provider) bash / node

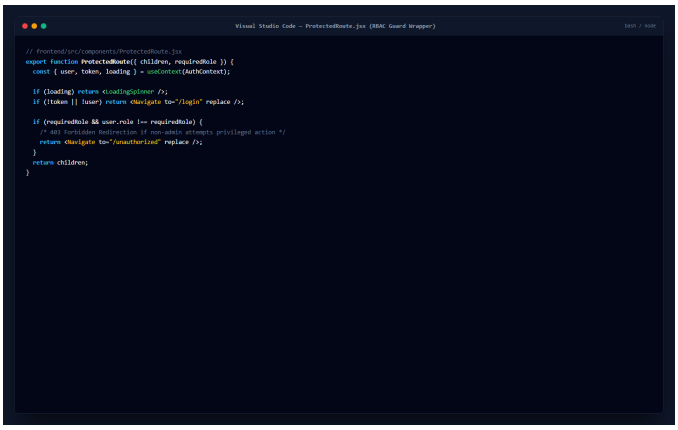
```jsx
// frontend/src/context/AuthContext.jsx
export const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(() => localStorage.getItem('truecert_token'));
  const [loading, setLoading] = useState(true);

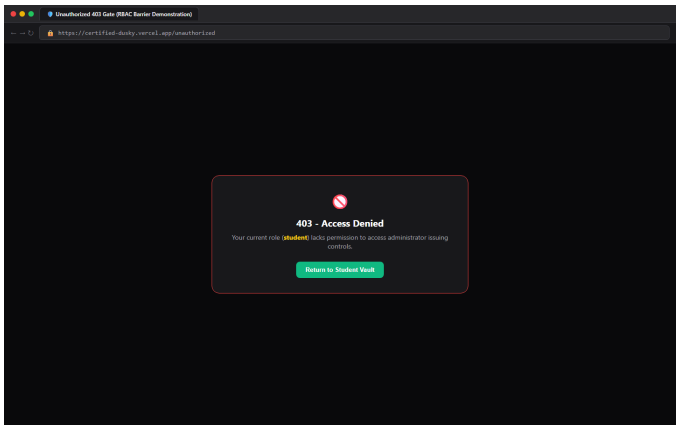
  useEffect(() => {
    /* Rehydrate stored session upon browser refresh */
    const savedUser = localStorage.getItem('truecert_user');
    if (token && savedUser) setUser(JSON.parse(savedUser));
    setLoading(false);
  }, [token]);

  const login = async (email, password) => {
    const res = await api.post('/api/auth/login', { email, password });
    localStorage.setItem('truecert_token', res.data.token);
    localStorage.setItem('truecert_user', JSON.stringify(res.data.user));
    setUser(res.data.user); setToken(res.data.token);
  };
}
```

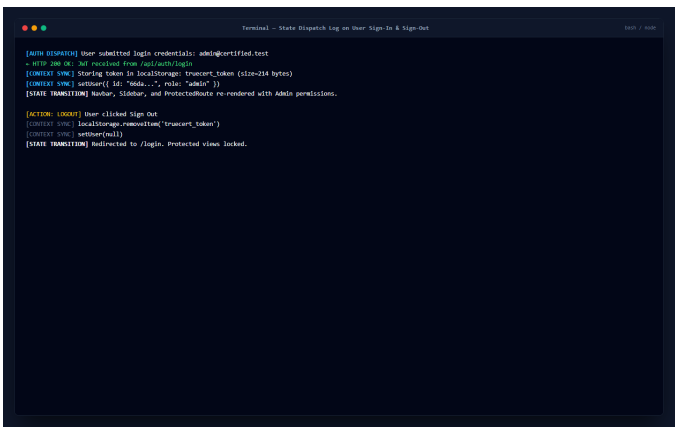
Screen 3.7: ProtectedRoute.jsx - RBAC Route Guard Component



Screen 3.8: Access Denied Gate - 403 Forbidden Redirection Interface



Screen 3.9: Terminal Dispatch Log - Sign-In & Sign-Out State Actions



Screen 3.10: React DevTools - AuthContext.Provider State Inspection

